Orbit Task Manager – Project 1

This document serves as the complete blueprint for Orbit Task Manager, a headless task management system. The backend exposes a RESTful API (Laravel), while the frontend is delivered as a Web SPA (Next.js) and a mobile app (Flutter) with identical feature sets.

---

1. Project Overview

Property	Value
Name	Orbit Task Manager
Architecture	Headless RESTful API (Laravel) + Web SPA (Next.js) + Mobile (Flutter)
Core Innovation	Multiple assignees per task (Many-to-Many between tasks and users)
Auth	Laravel Sanctum (token-based)
API Documentation	`dedoc/scramble` – auto-generated OpenAPI UI at `/docs/api`
No Real-time	WebSockets / Pusher are not used
No Tests	Automated testing (PHPUnit, Pest, Jest) is excluded
No Caching	Redis / Memcached are not used

2. Role-Based Permissions (with Multi-Assign)

Action	Admin	Editor	Developer
User Management (CRUD)	✅	❌	❌
Role Assignment	✅	❌	❌
Project Management (CRUD)	✅	✅	❌
Task Management (CRUD)	✅	✅	❌*
Assign multiple developers to task	✅	✅	❌
Update task status (if assigned)	✅	✅	✅
Add comments to tasks	✅	✅	✅
View Dashboard	All data	All data	Own tasks only

**Developers can view tasks, but cannot create, edit, or delete them.*

---

3. Database Schema

`users`

- `id`, `name`, `email`, `password`, `role` (`admin`, `editor`, `developer`), `timestamps`.

`projects`

- `id`, `name`, `description`, `status` (`active`, `on_hold`, `completed`), `created_by` (FK → `users.id`), `timestamps`.

`tasks`

- `id`, `title`, `description` (nullable), `project_id` (FK → `projects.id`), `created_by` (FK → `users.id`), `status` (`todo`, `in_progress`, `review`, `done`), `priority` (`low`, `medium`, `high`), `due_date` (nullable), `timestamps`.

`task_user`

- `id` (BIGINT), `task_id` (FK → `tasks.id` ON DELETE CASCADE), `user_id` (FK → `users.id` ON DELETE CASCADE), `created_at`, `updated_at`.

`task_comments`

- `id`, `task_id` (FK → `tasks.id`), `user_id` (FK → `users.id`), `content`, `timestamps`.

---

4. Development Timeline (4 Weeks)

Phase 0 – Setup & Environment (Days 1–2)

- Create GitHub repository and invite team members.
- Agree on API specification using Scramble (no Postman).
- Set up local development environments (Laravel Sail / Docker, Next.js, Flutter).

Phase 1 – Backend Core (Week 1) – *Laravel Team*

- **Authentication:** endpoints `api/register`, `api/login`, `api/logout`, `api/user` (Sanctum).
- **Role Middleware:** `admin`, `editor`, `developer` for protecting routes.
- **Multi-Assign Logic:** In Task Controller, handle arrays of `user_ids` using `sync()` or `attach()` when creating/updating tasks.
- **Scramble Integration:**
  - `composer require dedoc/scramble`
  - Annotate controllers with PHP attributes or OpenApi docblocks.
  - Interactive docs will be available at `/docs/api`.
- **Endpoints to deliver (example):**
  - `POST /api/tasks` – accepts `assigned_users[]` array.
  - `PUT /api/tasks/{id}` – accepts `assigned_users[]` array.
  - `GET /api/dashboard/stats` – Admin/Editor see global stats; Developer sees stats filtered by their assigned tasks.
  - `GET /api/users` – fetch list of all users (for dropdowns).

Phase 2 – Web Frontend (Week 2) – *Next.js Team*

The Web SPA must provide exactly the same pages and features as the Mobile App (Phase 3). Both should offer a seamless, role-aware experience.

2.1 Authentication Pages

- **Login** – email/password form.
- **Register** – name, email, password (admin and editor only, we may allow self-registration but roles are assigned by admin; we can restrict registration to admin-created accounts).
- **Logout** – clear token.

2.2 Dashboard

- **Stats Cards:**
  - Total projects, total tasks, tasks by status, tasks by priority.
  - For Developers: only projects/tasks they are assigned to are counted.
- **Recent Tasks** – list of latest tasks (filtered by role).
- **Quick Actions** – buttons to create new project / new task (visible to users with appropriate permissions).

2.3 Projects

- **List View** – paginated table/cards showing project name, status, created date.
- **Create Project** – modal or page with name, description, status.
- **Edit Project** – same fields pre-filled.
- **Project Details** – shows project info and a list of tasks belonging to that project (with status, priority, assigned developers). From here, users can click into a task.

2.4 Tasks

- **List View** – filterable by project, status, priority, assigned to (multi-select).
- **Create Task** – form with:
  - Title, description, project (dropdown), priority, due date.
  - Multi-select for developers – a dropdown (e.g., React-Select) allowing selection of multiple users from the `users` list (only those with `developer` role are shown).
- **Edit Task** – same fields pre-filled, with the multi-select showing currently assigned users.
- **Task Detail** – shows all task info, assigned developers list, and a comment section:
  - Add comment (textarea + submit).
  - Display comments with author name and timestamp.
- **Status Update** – dropdown to change status. Enabled for:
  - Admin / Editor (always).

- Developer (only if they are among the assignees).

2.5 User Management (Admin only)

- **List Users** – table with name, email, role.
- **Create/Edit User** – form with name, email, password, role.

Phase 3 – Mobile App (Week 3) – *Flutter Team*

The mobile app must be feature-equivalent to the Web SPA. Every page and action listed above must be implemented in Flutter, with the same business logic and permissions.

3.1 Screens (identical to Web)

- **Authentication:** Login, Register, Logout.
- **Dashboard:** Stats cards and recent tasks.
- **Projects:** List, Create, Edit, Detail (with tasks list).
- **Tasks:** List (with filters), Create, Edit, Detail (with comments and status update).
- **User Management (admin only):** List, Create, Edit.

3.2 Key UI Components

- **Multi-select for developers:** Use a `CheckboxListTile` or `Chip`-based selection widget.
- **Dropdowns** for status, priority, project.
- **Comment input** with a text field and submit button.

Phase 4 – Finalization & Deployment (Week 4) – *All Teams*

- **Deployment to awardspace.net** (shared hosting):
  - See detailed steps below.
- **Mobile App Deployment**:
  - Deploy APK.

---

5. Environment Variables & CORS

Frontend	Env Variable
Next.js	`NEXT_PUBLIC_API_URL=https://your-domain.awardspace.net/api`
Flutter	`BASE_URL` (set in constants file)

CORS Configuration (Laravel `config/cors.php`):

- `paths` = `['api/*']`
- `allowed_origins` = include your Next.js deployment URL and Flutter test ports (e.g., `http://localhost:3000`, `http://localhost:8080`).

---

6. API Documentation (Scramble)

- UI available at: `https://your-domain.awardspace.net/docs/api`
- JSON spec at: `https://your-domain.awardspace.net/docs/api.json`
- No manual export/import – automatically generated from code annotations.

---

7. Final Notes

- **Consistency:** Web and Mobile have identical features; any future feature addition must be implemented on both.